INTERVIEW PREP

System Design (Interview Prep)

A visual, framework-driven playbook for designing scalable systems and acing the system design interview — from estimation and building blocks to CAP, API design, and ML/LLM architectures.

system-design · scalability · distributed-systems · caching · databases · interview-prep · architecture

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. The Interview Framework
2. Back-of-Envelope Estimation
3. Load Balancers
4. Caching
5. Databases: SQL, NoSQL, and the CAP Theorem
6. Message Queues & Event Streaming
7. CDN & Rate Limiting
8. API Design
9. Consistency Models
10. Scalability Patterns
11. Worked Example: URL Shortener
12. ML/LLM System Design
13. Building Blocks Cheat Sheet
14. Trade-offs Reference
15. Q&A Practice
16. How to Communicate Your Design

The Interview Framework

Every system design interview follows the same skeleton. Internalise this six-step arc and you will never lose the thread — even under pressure.



The six-step interview framework — work left to right, narrate each transition aloud.

Step 1 — Clarify Requirements

Never design until you ask. Spend 3–5 minutes surfacing functional and non-functional requirements. A missed assumption wastes 20 minutes of deep-dive on the wrong component.

FUNCTIONAL

What does the system *do*? Core user actions (write URL → get short URL; post tweet → timeline appears). Scope: read vs write flows, admin flows, mobile clients?

NON-FUNCTIONAL

Latency targets (p99 < 100 ms?), availability SLA (99.9% or 99.99%?), consistency model (strong vs eventual), geo distribution, security, compliance (GDPR?), cost sensitivity.

Step 2 — Estimate Scale

Back-of-envelope numbers anchor every architectural decision. Compute QPS, storage, and bandwidth before drawing a single box.

Step 3 — High-Level Design

Draw 5–8 components: clients, API gateway/LB, app servers, primary store, cache, async queue, CDN. Show data flow with arrows. Name the components clearly. Check in with the interviewer before going deep.

Step 4 — Deep-Dive

The interviewer will guide you to the hardest sub-problems: DB schema, consistency protocol, write path, failure handling. Lead with the most critical path. Speak complexity aloud.

Step 5 — Trade-offs

Every choice has a cost. SQL → strong consistency but limited horizontal scale. Kafka → high throughput but eventual delivery. Say "I chose X because Y, accepting Z." Interviewers reward trade-off reasoning over memorised answers.

Step 6 — Wrap-Up

Name 2–3 bottlenecks you haven't addressed (hot partitions, cache stampede, single-region fail). Suggest what to solve next. This signals engineering maturity.

> ✦ **Pro tip**
> Draw on paper/whiteboard while narrating. An interviewer following your diagram retains it 3× better than one just listening.

Back-of-Envelope Estimation

Capacity math separates candidates who can design real systems from those who memorise patterns. Practise until the numbers feel intuitive.

Key Numbers to Memorise

Metric	Rule of Thumb	Notes
1 million users, active 10%	100 K DAU	Adjust for peak vs avg
1 request/user/day	~1.2 QPS	100 K req / 86 400 s
1 KB per record	1 M records = 1 GB	Simple maths
Twitter-scale writes	~6 000 tweets/s peak	500 M tweets/day
SSD read latency	~100 µs	vs HDD 10 ms
Network round-trip (same DC)	~0.5 ms	Cross-continent ≈ 150 ms
L1 cache hit	~1 ns	L2 ≈ 5 ns, RAM ≈ 100 ns

Estimation Template

QPS

Peak QPS ≈ 2–3× average. Average = total requests per day ÷ 86 400. State read/write ratio (often 10:1 to 100:1).

STORAGE

Daily data = records/day × avg record size. Multiply by retention years. Add 20% overhead for indices, replication factor (usually 3).

BANDWIDTH

Ingress = write QPS × payload size. Egress = read QPS × response size. CDN can offload 80–95% of egress for media-heavy services.

Worked estimation: URL shortener at 100 M DAU

> **Note**
> Estimation is about order of magnitude, not precision. Being off by 2× is fine. Reveal your assumptions explicitly — "assuming 100:1 read/write, roughly 10:1 peak/average…"

Load Balancers

A load balancer is the first distributed systems primitive. Know the layers, algorithms, and when sticky sessions matter.

L4 vs L7

LAYER 4 (TRANSPORT)

Routes by IP + TCP port. Extremely fast — no HTTP parsing. Cannot inspect URL path, headers, or cookies. Used for raw throughput (databases, gRPC). Examples: AWS NLB, HAProxy in TCP mode.

LAYER 7 (APPLICATION)

Routes by HTTP method, path, headers, cookies. Enables path-based routing (`/api` → service A, `/static` → CDN), SSL termination, health checks, WebSocket upgrades. Examples: AWS ALB, Nginx, Envoy.

Routing Algorithms

Algorithm	Best For	Weakness
Round-Robin	Homogeneous servers, stateless	Ignores server load
Least Connections	Variable request duration	Slight overhead tracking connections
IP Hash	Sticky sessions (stateful)	Uneven distribution if few clients
Weighted Round-Robin	Heterogeneous server capacity	Manual weight tuning
Least Response Time	Latency-sensitive APIs	More complex health tracking
Random	Simple, low overhead	Not optimal; use for testing

> ✦ **Interview tip**
> When the interviewer asks "how do you handle server failures?" — mention health checks (active probes every 5 s, passive circuit-breaking), automatic draining of in-flight connections, and auto-scaling triggers.

Health Checks & High Availability

LBs themselves are SPOFs — use an active-passive pair with a floating VIP (Virtual IP) and VRRP/ECMP. In cloud environments (AWS, GCP) the load balancer is a managed multi-AZ service and this is handled for you.

Caching

Caching is the single highest-leverage optimisation in most systems. Know the strategies, eviction policies, and failure modes cold.

Where to Cache

CLIENT / BROWSER

Cache-Control headers, ETags. Zero server load for static assets. TTL-driven invalidation.

CDN EDGE

Static + semi-static content served globally. Origin shield reduces origin traffic by 80–95%.

IN-PROCESS MEMORY

Fastest (nanoseconds). Limited to one server. Use for reference data — country codes, config flags.

DISTRIBUTED CACHE

Redis / Memcached shared across app servers. Millisecond latency. Central invalidation. 100 K–1 M+ ops/s per node.

DB QUERY CACHE

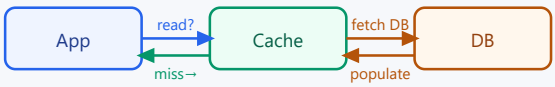
Deprecated in MySQL 8+. Prefer application-level caching — more control, less contention.

READ REPLICAS

Not a cache per se, but offloads read traffic. Eventual consistency — replicas lag by milliseconds.

Cache Strategies

Cache-Aside (Lazy Loading)



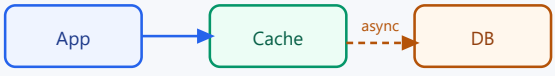
\+ Simple. Only caches what is used. — Cache miss = 3 round trips. Stale on write.

Write-Through



\+ Cache always fresh. — Write latency doubles. Wasted writes for rarely-read data.

Write-Back (Write-Behind)



\+ Lowest write latency. Batched DB writes. — Data loss risk if cache crashes before flush.

Three caching strategies — Cache-Aside is the default; Write-Through for read-heavy data that must be fresh; Write-Back for high-write, latency-sensitive workloads.

Eviction Policies

Policy	Rule	Use When
LRU (Least Recently Used)	Evict the key unused longest	General purpose — the safe default
LFU (Least Frequently Used)	Evict the key hit least often	Long-lived hot keys (song catalogue)
TTL (Time-to-Live)	Expire after fixed duration	Session tokens, OTP codes
FIFO	Evict oldest inserted key	Queue-style, rare in caching
Random	Evict random key	Approximates LRU cheaply
allkeys-lru (Redis)	LRU across all keys, no TTL required	When TTLs are inconsistently set

Cache Failure Modes

CACHE STAMPEDE

Many threads miss simultaneously and all hit the DB. Fix with probabilistic early expiry, a mutex/lock on the first miss (cache aside with locks), or background refresh.

CACHE PENETRATION

Queries for keys that never exist bypass the cache and hit the DB every time. Fix with a Bloom filter at the cache layer to reject known-absent keys.

CACHE AVALANCHE

Many keys expire at the same time (e.g., after a cache flush). Fix with jittered TTLs — add ±10–20% random offset to each key's TTL.

Redis vs Memcached

Feature	Redis	Memcached
Data structures	String, Hash, List, Set, ZSet, Stream, Bitmap, HLL	String only
Persistence	RDB snapshots + AOF log	None (RAM only)
Replication	Primary-replica + Sentinel/Cluster	Client-side sharding only
Lua scripting	Yes	No
Best for	Leaderboards, sessions, pub/sub, rate limiting, distributed locks	Simple object cache, multi-threaded high throughput

Databases: SQL, NoSQL, and the CAP Theorem

Choosing the right database is the most consequential early decision. Understand the trade-offs, not just the names.

SQL vs NoSQL — When to Use Which

RELATIONAL (SQL)

Choose when: data has complex relationships, transactions span multiple tables (financial transfers, inventory), schema is stable, strong consistency is required. PostgreSQL is the modern default. MySQL for legacy systems. CockroachDB / Spanner for distributed ACID.

NOSQL

Choose when: massive scale (hundreds of millions of rows), flexible/evolving schema, high write throughput, simple access patterns (key-value, document). Cassandra for write-heavy time-series; MongoDB for document; DynamoDB for serverless key-value; Elasticsearch for full-text search.

Sharding / Partitioning

Sharding splits a dataset across multiple DB nodes. Each shard owns a range of keys. It is the primary path to horizontal write scaling.

Strategy	How	Pros	Cons
Hash Sharding	shard = hash(key) % N	Even distribution	Range queries span all shards; resharding is painful
Range Sharding	shard by key range (A–M, N–Z)	Range queries efficient	Hot partition if range is uneven (time-series: latest shard gets all writes)
Directory / Lookup	Mapping table: key → shard ID	Flexible; easy resharding	Lookup table is a SPOF; extra hop
Geo-Sharding	shard by region	Data locality, compliance	Cross-region queries slow

> ⚠ **Warning**
> Avoid cross-shard transactions whenever possible — they require distributed locks (2PC) and can degrade to O(n) latency. Design your data model to co-locate related records on the same shard.

Replication

SINGLE-LEADER

One primary accepts writes; replicas stream changes. Reads can go to replicas (replication lag ≈ ms). Most common. Failover: replica promoted on primary death.

MULTI-LEADER

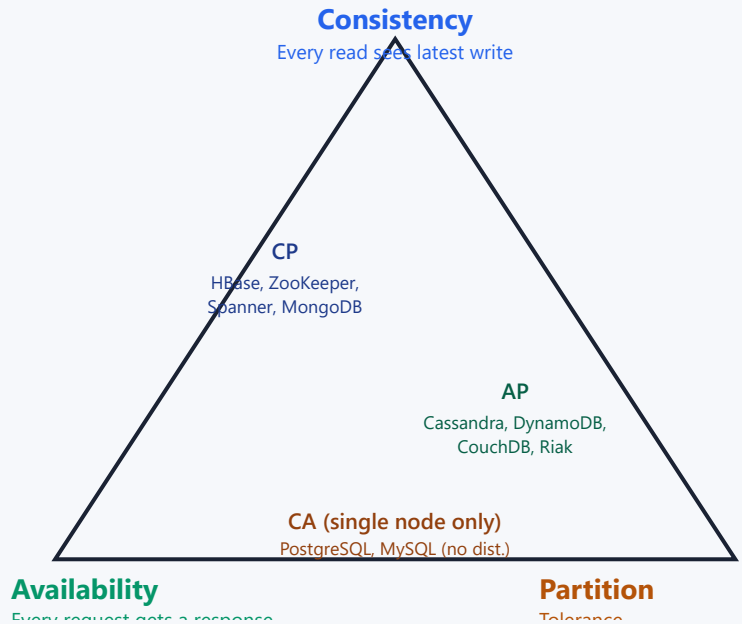
Multiple nodes accept writes (geo-distributed teams, offline clients). Requires conflict resolution (last-write-wins, CRDTs, custom merge). Complexity cost is high.

LEADERLESS (DYNAMO-STYLE)

Any node accepts reads/writes. Quorum: W + R > N for consistency. Used in Cassandra, DynamoDB. Tunable consistency. Best for high availability.

The CAP Theorem

A distributed system can guarantee at most two of the three: Consistency, Availability, Partition Tolerance. Because network partitions are inevitable, you always choose between CP and AP.



CAP Triangle — real distributed systems sit on the CP or AP edge. CA is only possible without network partitions, i.e. single-node.

> **Note**
> PACELC extends CAP: even without a partition (E), systems trade off Latency (L) vs Consistency (C). DynamoDB defaults to low latency with eventual consistency; it can be tuned toward consistency at the cost of latency.

Database Indexing Essentials

B-tree indexes (default in PostgreSQL/MySQL) excel at range queries and equality. Composite indexes follow left-prefix rules. Hash indexes suit exact equality only. Avoid over-indexing — each index adds write overhead and storage. Cover your most common query patterns; EXPLAIN ANALYZE every slow query.

Message Queues & Event Streaming

Async communication decouples services, absorbs traffic spikes, and enables durable processing. Know when to use a queue vs a stream.

Queue vs Stream

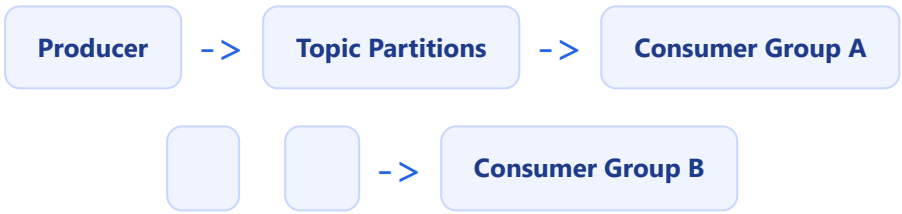
MESSAGE QUEUE (RABBITMQ, SQS)

Each message consumed once by one consumer. Message deleted on ACK. Best for task distribution, background jobs (send email, resize image). Supports fan-out via exchanges. Delivery guarantees: at-least-once or exactly-once (with transactions).

EVENT STREAM (KAFKA, KINESIS)

Log-based. Messages retained for days/weeks. Multiple consumer groups each read independently. Best for audit logs, analytics pipelines, CDC (Change Data Capture), event sourcing. Replay from any offset.

Kafka Architecture



Kafka partitions enable parallel consumption. Within a partition messages are ordered. A consumer group scales horizontally up to the partition count — beyond that, consumers are idle. Key-based partitioning ensures ordering for related events (all events for user ID X go to the same partition).

Concept	Kafka	RabbitMQ
Model	Persistent log / stream	Message queue (push)
Retention	Days to weeks (configurable)	Until consumed/ACKed
Ordering	Per-partition	Per-queue (FIFO)
Throughput	Millions/s per partition	Tens of thousands/s
Consumer model	Pull (consumer controls offset)	Push (broker delivers)
Replay	Yes — seek to any offset	No — message gone after ACK
Best for	Event sourcing, analytics, CDC	Task queues, RPC, work distribution

Tip

When asked "how do you handle a spike in user uploads?" the answer is: write upload metadata to a Kafka topic; a downstream consumer pool processes them asynchronously. The API responds immediately; the queue absorbs the burst.

CDN & Rate Limiting

Content Delivery Network (CDN)

A CDN is a globally distributed cache of static (and increasingly dynamic) content. Edge nodes serve requests from the geographically nearest PoP (Point of Presence), reducing latency from hundreds of milliseconds to single-digit milliseconds.

PULL CDN

CDN fetches from origin on first cache miss, then caches. Simple to operate. Cold start latency on first request per edge. Examples: Cloudflare, Fastly.

PUSH CDN

You upload content to CDN nodes explicitly. Good for rarely-changing large files. Must manage cache invalidation. Examples: AWS CloudFront with S3 sync.

Use CDNs for: images/video, JavaScript/CSS bundles, HTML pages with long TTLs, API responses that are the same for all users (public product listings). Do not cache: personalised API responses, session-sensitive data.

Rate Limiting

Rate limiting protects services from abuse, runaway clients, and DDoS. Implement at the API gateway or a dedicated reverse proxy layer.

Algorithm	How	Pros	Cons
Token Bucket	Bucket fills at rate r. Each request costs 1 token.	Allows bursting up to bucket size	More state per user
Leaky Bucket	FIFO queue drains at constant rate	Smooth output — no bursts	Latency for queued requests
Fixed Window Counter	Count requests per window (1 min). Reset at boundary.	Simple, O(1) storage	Boundary spike: 2× traffic at window edge
Sliding Window Log	Timestamp log of requests; prune old entries	Accurate, no boundary spike	High memory — stores every request timestamp
Sliding Window Counter	Weighted blend of current + previous window	Accurate, low memory	Approximate (not exact)

> **Distributed Rate Limiting**
> Token counters stored in Redis with Lua scripts or atomic INCR+EXPIRE. Use Redis Cluster for HA. In multi-region setups, counters can be per-region (approximate global limit) or centralised (exact but higher latency).

API Design

API design is an interface contract. In system design interviews, briefly discuss the client-facing API before designing internals.

REST

Resource-oriented, stateless, HTTP verbs (GET/POST/PUT/PATCH/DELETE). Standard pagination: cursor-based (stable) or offset-based (simple but inconsistent on live data). Use HTTP status codes semantically: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 429 Too Many Requests, 503 Service Unavailable. Version via URL (`/v1/`) or `Accept` header.

gRPC

Protocol Buffers (binary serialisation), HTTP/2, bidirectional streaming, auto-generated clients. 5–10× smaller payload than JSON. Best for internal service-to-service communication. Requires HTTP/2 support from load balancers.

GraphQL

Client specifies exact fields needed. Eliminates over-fetching and under-fetching. Single endpoint. Trade-off: N+1 query problem mitigated by DataLoader (batching), but can be expensive server-side without query depth limits and cost analysis.

WebSocket

Full-duplex persistent connection. Use for: real-time chat, live dashboards, collaborative editing, notifications. Stateful — session is pinned to one server. Use IP-hash LB or move session state to Redis pub/sub for horizontal scaling.

API Gateway Pattern



The API gateway handles: authentication/authorisation, rate limiting, request routing, SSL termination, protocol translation (REST → gRPC), response aggregation, logging and tracing injection. Examples: AWS API Gateway, Kong, Nginx, Envoy.

Consistency Models

Distributed systems trade consistency for availability and latency. Know the spectrum — from strict serialisability to eventual consistency.

Model	Guarantee	Latency	Example
Strict Serialisability	Total order; reads see latest write globally	Highest	Spanner, FoundationDB
Linearisability	Every operation appears instantaneous; real-time order	High	Zookeeper, etcd
Sequential Consistency	All processes see same order; not necessarily real-time	Medium	Raft consensus
Causal Consistency	Causally related writes seen in order by all clients	Medium	MongoDB sessions
Read-Your-Writes	A client always sees its own writes	Low	Sticky sessions or token routing
Eventual Consistency	All replicas converge given no new writes	Lowest	DynamoDB default, DNS

✦ **Interview answer scaffold**

"For user profile reads after a write, I need read-your-writes consistency. I'll route all reads for a user to the same replica for 5 seconds after a write using a session token. For the global feed, eventual consistency is fine — a few seconds of staleness is invisible to users."

Achieving Read-Your-Writes at Scale

Three practical patterns: (1) route all requests for a user to the primary for N seconds after a write; (2) attach a "replication token" (write LSN) to the response and route reads to a replica only if its LSN ≥ token; (3) write to cache simultaneously on DB write so reads always find the latest version in cache.

Scalability Patterns

Scale by dimension: vertical (bigger box), horizontal (more boxes), or by sharding data and distributing state.

Vertical vs Horizontal Scaling

VERTICAL (SCALE-UP)

Larger CPU, more RAM, faster SSD. Simple — no code changes. Hard ceiling (largest instance type). Single point of failure. Good for databases that resist horizontal sharding (complex joins, ACID transactions). Cost: diminishing returns beyond a threshold.

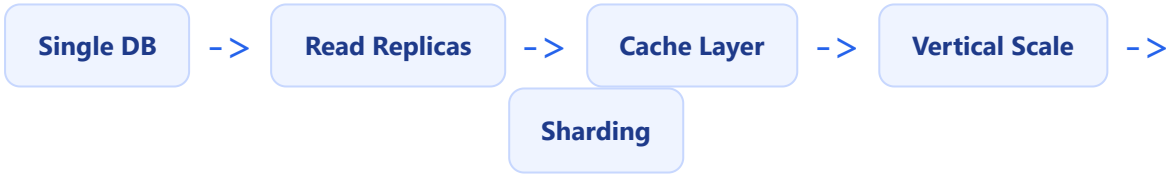
HORIZONTAL (SCALE-OUT)

More servers behind a load balancer. Linear cost scaling. Requires stateless application tier — all state lives in shared storage (DB, Redis, object store). Fault tolerant — losing one node is not catastrophic. Required for >1 TB data or >100 K QPS sustained.

Stateless Services

A service is stateless when any instance can handle any request. Session state, user context, and ephemeral data must be externalised to Redis/Memcached (sessions), S3/GCS (file uploads), or a DB (persistent state). This is non-negotiable for horizontal scaling and zero-downtime deploys.

Database Scaling Ladder



Exhaust cheaper options before sharding. Sharding multiplies operational complexity: queries must be shard-aware, joins across shards are expensive, migrations require careful coordination.

Additional Patterns

Pattern	Problem Solved	Trade-off
CQRS (Command/Query Segregation)	Separate read and write models for different scaling needs	Two models to maintain; eventual consistency between them
Event Sourcing	Audit log, replay, time-travel debugging	Complex queries on event log; snapshot needed for performance
Saga Pattern	Distributed transactions across microservices	Compensating transactions complexity; eventual consistency
Circuit Breaker	Prevent cascade failures when downstream service is slow	Caller must handle "open circuit" state gracefully
Bulkhead	Isolate failures to a pool — one slow service doesn't starve others	Over-provisioning thread pools
Backpressure	Slow producers/consumers from overwhelming downstream	Requires queue depth monitoring and producer throttling

Worked Example: URL Shortener

A URL shortener is the "hello world" of system design. Walk through the full design — estimation, schema, read/write paths, and failure handling.

Requirements

FUNCTIONAL

- POST /shorten — accepts long URL, returns short code
- GET /{code} — redirects to original URL (301/302)
- Optional: custom aliases, expiry, click analytics

NON-FUNCTIONAL

- 100:1 read/write ratio
- Redirect latency p99 < 50 ms
- 99.99% availability for redirects
- Short codes: globally unique, hard to predict

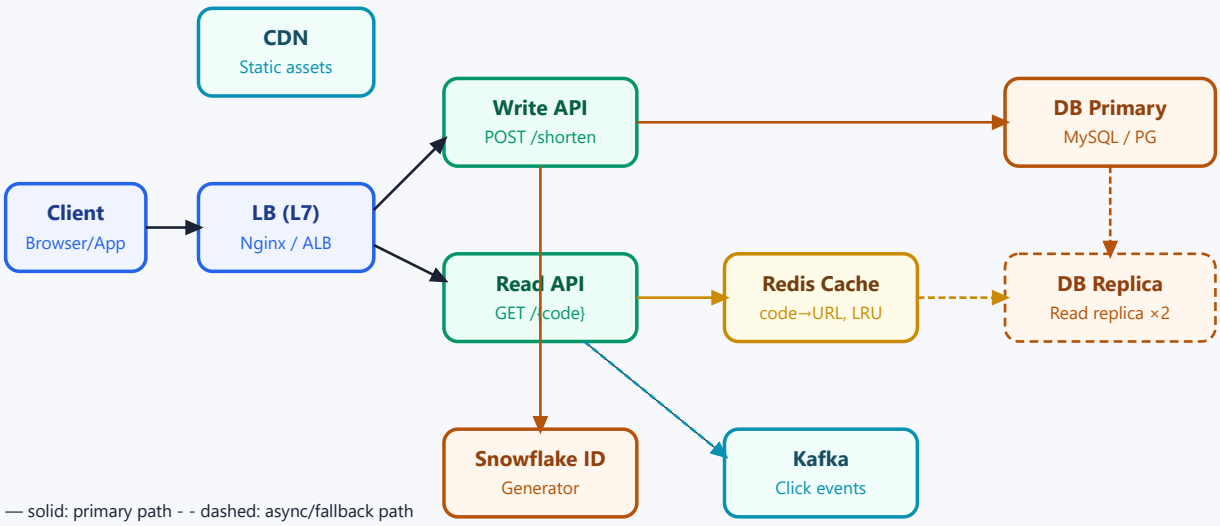
Estimation

100 M DAU, 1% create URLs → 1 M writes/day ≈ 12 writes/s. Reads: 100:1 → 1 200 reads/s average, 3 600 reads/s peak. Storage: 1 M × 365 × 5 years × 500 B ≈ 900 GB. Bandwidth: trivial (HTTP redirects, no payload).

Short Code Generation

Options: (1) **Random base62 (7 chars)** — 62^7 = 3.5 T URLs, collision probability low, but must check DB on collision; (2) **Snowflake-style auto-increment ID → base62** — monotonically increasing, no collision check needed, reveals insertion order; (3) **MD5/SHA hash of URL truncated** — deterministic, deduplication free, but rare collision. Best answer: Snowflake ID (distributed, time-ordered, no collisions) encoded in base62 → 7 chars.

Architecture Diagram



URL Shortener architecture — write path (blue) vs read path (gold) with Redis cache-aside, read replicas, Kafka analytics stream, and a distributed Snowflake ID generator.

Read Path

GET /abc1234 -> **LB** -> **Read API** -> **Redis (hit?)** -> **302 Redirect**

On cache miss: Read API → DB Replica → populate Redis (TTL 24 h) → return 302. Click event published to Kafka asynchronously. Cache hit rate should be >95% for popular URLs; tail latency dominated by DB replica reads.

Write Path

POST /shorten -> **Write API** -> **Snowflake ID** -> **base62 encode** -> **DB Primary** -> **Return short URL**

DB Schema

SQL

```sql
CREATE TABLE url_map (
  id          BIGINT PRIMARY KEY,         -- Snowflake ID
  short_code  VARCHAR(10) NOT NULL UNIQUE, -- base62(id)
  long_url    TEXT        NOT NULL,
  user_id     BIGINT,
  created_at  TIMESTAMP   DEFAULT NOW(),
  expires_at  TIMESTAMP,
  click_count BIGINT      DEFAULT 0        -- updated by analytics consumer
);
CREATE INDEX idx_short_code ON url_map (short_code);
```

Failure Scenarios

Failure	Impact	Mitigation
Cache node dies	All reads hit DB — 10× load spike	Redis Cluster (multi-node); circuit breaker; serve stale on error
DB Primary dies	Writes fail; reads can continue from replica	Automated failover (RDS Multi-AZ, Patroni); writes queue in Kafka
Short code collision	Second URL overwrites first	Snowflake IDs are unique by design; unique constraint in DB
Hot URL (viral)	Single key hammers cache, then DB	Local in-process cache per app server for top-N keys; cache-aside with lock

ML/LLM System Design

AI/ML workloads have unique infrastructure concerns: GPU affinity, large model weights, embedding indices, and streaming inference. Interviewers increasingly expect fluency here.

ML Model Serving Layer

ONLINE SERVING (LOW LATENCY)

Real-time inference. Framework: TorchServe, Triton Inference Server, vLLM (for LLMs), ONNX Runtime. GPU instances (A10G/A100/H100). Batching: dynamic micro-batching (group requests in a 5–20 ms window). Auto-scaling on GPU utilisation + request queue depth.

BATCH / OFFLINE SERVING

Bulk predictions — product recommendations, email content scoring, document classification. Spark / Ray for distributed processing. GPU spot instances for cost. Results written to a feature store or DB for downstream reads.

LLM Serving Optimisations

Technique	What It Does	Impact
KV Cache	Caches Key/Value attention tensors for the prompt prefix	Eliminates redundant prompt re-computation; critical for multi-turn chat
Continuous Batching	Interleave tokens from multiple requests mid-generation	3–10× GPU utilisation improvement vs naive batching
Speculative Decoding	Small draft model proposes tokens; large model verifies	2–4× throughput for low-temperature generation
Quantisation (INT8/INT4)	Reduce weight precision	2–4× memory reduction; slight quality trade-off
Tensor Parallelism	Shard weight matrices across GPUs	Fit larger models; latency scales sub-linearly
Prefix Caching (vLLM)	Re-use KV cache across requests with identical system prompts	Dramatically reduces TTFT for shared-system-prompt workloads

RAG Pipeline at Scale



EMBEDDING SERVICE

Dedicated embedding endpoint (e.g. text-embedding-3-small, GTE-large). Batch offline for document corpus. GPU for throughput; CPU for p99 latency with small models. Cache embeddings to avoid re-embedding identical chunks.

VECTOR DATABASE

Stores dense vectors + metadata. ANN algorithms: HNSW (low latency, high recall), IVF-PQ (compression + speed). Options: Pinecone, Weaviate, Qdrant, pgvector (PostgreSQL). Scale: 100 M vectors needs dedicated vector DB, not in-process FAISS.

RETRIEVAL QUALITY

Hybrid search: dense (semantic) + sparse (BM25 keyword) with RRF re-ranking. Chunk strategy: 256–512 token chunks with 10–20% overlap. Re-ranker (cross-encoder) re-orders top-K before sending to LLM. Metadata filters reduce search space.

Feature Store

A feature store centralises ML features so they are consistent between training and serving — eliminating training/serving skew. Offline store (S3 + Spark) for historical features. Online store (Redis/DynamoDB) for low-latency feature retrieval at serve time. Examples: Feast, Tecton, Vertex AI Feature Store.

ML System Design Checklist

Checklist

- Define the ML task (classification, regression, ranking, generation)
- Feature pipeline: sources → transform → online/offline store
- Training: batch size, distributed training (DDP/FSDP), experiment tracking (MLflow, W&B)
- Evaluation: offline metrics (AUC, NDCG, BLEU) vs online metrics (CTR, revenue)
- Serving: online (latency < 100 ms) vs batch; model registry → canary deploy
- Monitoring: data drift, concept drift, model performance decay, alerting
- Feedback loop: user signals → labels → retraining pipeline

Building Blocks Cheat Sheet

One-page reference for the most common components and the questions they answer.

Component	Primary Role	Go-To Options (2026)	Key Config / Tuning
L7 Load Balancer	Route HTTP traffic, TLS termination	AWS ALB, Nginx, Envoy, Traefik	Health check interval, sticky sessions, connection draining
Reverse Proxy / API GW	Auth, rate limit, routing	Kong, AWS API GW, Apigee	Rate limit policy, JWT validation, plugin chain
Distributed Cache	Sub-ms reads, offload DB	Redis 7 (Cluster), Memcached	Max memory policy (allkeys-lru), eviction, persistence RDB+AOF
RDBMS	ACID transactions, complex queries	PostgreSQL 17, MySQL 9, CockroachDB	Connection pool (PgBouncer), read replicas, EXPLAIN ANALYZE
Wide-Column / KV NoSQL	High write throughput, simple access	Cassandra 5, DynamoDB, ScyllaDB	Replication factor, consistency level, compaction strategy
Document DB	Flexible schema, rich queries	MongoDB 8, Firestore	Indexes, aggregation pipeline, change streams
Search Engine	Full-text search, faceted filter	Elasticsearch 9, OpenSearch	Index mapping, shard count, refresh interval
Vector DB	Semantic search, RAG retrieval	Qdrant, Weaviate, pgvector	HNSW ef_construction, index segment merge, hybrid BM25+dense
Message Queue	Async task dispatch, decoupling	RabbitMQ 4, AWS SQS	Dead-letter queue, visibility timeout, message TTL
Event Stream	Durable log, multi-consumer	Kafka 3.9 (KRaft), AWS Kinesis	Partition count, replication factor, consumer lag alert
Object Storage	Blob / file storage at scale	AWS S3, GCS, Azure Blob	Multipart upload, lifecycle rules, presigned URLs
CDN	Static asset delivery, latency	Cloudflare, AWS CloudFront, Fastly	Cache-Control TTL, origin shield, purge/invalidation strategy
LLM Inference	Text generation, embeddings	vLLM, Triton, TGI, Bedrock	Continuous batching, KV cache size, quantisation, TP degree

Trade-offs Reference

Use this table to justify architectural decisions. Every choice is a trade-off — internalise the vocabulary.

Decision	Option A	Option B	When to Choose A	When to Choose B
Consistency	Strong (synchronous replication)	Eventual (async replication)	Financial transactions, inventory, leaderboards	Social feed, analytics, DNS, notifications
DB type	SQL (PostgreSQL)	NoSQL (Cassandra)	Complex joins, ACID needed, stable schema	100 M+ rows, write-heavy, simple key-based access
Cache strategy	Cache-Aside	Write-Through	Read-heavy, tolerate brief staleness	Write-heavy read-back, freshness critical
Scaling axis	Vertical	Horizontal	Stateful DB under 10 TB, strong consistency, simplicity	Stateless app tier, >100 K QPS, fault tolerance needed
Async comms	Message Queue (SQS)	Event Stream (Kafka)	Task distribution, each message consumed once	Event log, multiple consumers, replay needed
API protocol	REST (JSON)	gRPC (Protobuf)	Public APIs, browser clients, simplicity	Internal microservices, high throughput, streaming
ID generation	UUID v4 (random)	Snowflake (time-ordered)	Fully decentralised, opaque IDs needed	DB index locality, time-sortable, smaller than UUID
Storage	SSD (block)	Object store (S3)	Low latency DB I/O, random access	Large blobs, images, backups, low cost per GB

Q&A Practice

Drill these out loud. The ability to articulate concise answers under pressure is the interview skill.

How would you design Twitter's timeline?

Two approaches: (1) Fan-out on write — on tweet, push tweet ID to all followers' timeline caches in Redis. O(followers) write amplification; bad for celebrities. (2) Fan-out on read — merge timelines from followed users at read time. O(following) DB reads per page load. Hybrid: fan-out on write for <1 M followers; fan-out on read for celebrities. Cache precomputed timelines in Redis sorted sets (ZRANGEBYSCORE by timestamp).

How do you prevent a cache stampede?

Three patterns: (1) Probabilistic early expiry — before TTL expires, a small probability of refreshing on each read, so many callers don't all expire at once; (2) Mutex/lock on first miss — first thread to miss acquires a lock, fetches DB, populates cache; other threads wait and read from cache once populated; (3) Background refresh — a daemon refreshes keys proactively before TTL, so the cache never actually expires for hot keys.

Explain the difference between a message queue and an event stream.

A message queue (SQS, RabbitMQ) delivers each message to one consumer, deletes on ACK. An event stream (Kafka, Kinesis) is a durable log — messages are retained, and multiple independent consumer groups each read at their own offset. Use queues for task distribution; use streams for event sourcing, audit logs, and multi-consumer fanout.

When would you shard a database?

Only after exhausting: read replicas, caching, vertical scaling, and query optimisation. Sharding is warranted when: single-node write throughput is saturated (>~50 K writes/s), or data size exceeds single-node storage (~10 TB). Sharding adds complexity: cross-shard queries, distributed transactions, and resharding pain. Choose shard key carefully — avoid hot partitions; co-locate frequently joined data.

What is the CAP theorem and how does it guide database choice?

CAP: Consistency (all nodes return the same data), Availability (every request returns a response), Partition Tolerance (the system operates despite network splits). Since partitions are unavoidable in distributed systems, you choose CP (PostgreSQL, ZooKeeper — sacrifice availability under partition) or AP (Cassandra, DynamoDB — remain available, sacrifice strong consistency). For financial data: CP. For social feeds: AP with read-your-writes for self-reads.

How do you design a rate limiter at scale?

Use a Token Bucket algorithm (allows bursts) or Sliding Window Counter (accurate, low memory). Store counters in Redis with INCR+EXPIRE using a Lua script for atomicity. For multi-region: each region maintains a local counter (approximate global limit) or sync to a central Redis (exact but adds latency). Return 429 with Retry-After header. Apply limits at the API gateway before hitting application servers.

How does consistent hashing work and why is it used for sharding?

Nodes and keys are mapped to a hash ring (0 to 2^{32}). A key is owned by the first node clockwise from its hash. When a node is added/removed, only the keys between the new node and its predecessor move — typically 1/N of all keys, not all keys. This minimises data movement during scale-out events. Virtual nodes (vnodes) per physical node smooth key distribution and handle heterogeneous capacity.

How would you design a notification system for 100 M users?

Publisher writes notification event to Kafka. Fan-out service reads topic, looks up user device tokens, groups by channel (push/email/SMS). Channel workers (one per channel type) batch-send via Firebase FCM, AWS SES, Twilio. Store notification records in Cassandra (wide-column, write-heavy). Unread badge count in Redis INCR/DECR. For real-time: WebSocket gateway subscribes to a user-specific Redis pub/sub channel; notification event triggers push to connected clients.

What is read-your-writes consistency and how do you achieve it?

Read-your-writes guarantees that after you write, subsequent reads from the same session see your write. Strategies: (1) Route all reads to the primary for N seconds post-write; (2) Attach the write's LSN (log sequence number) to the response cookie; route reads to a replica only if replica_lsn ≥ write_lsn; (3) Populate the cache on write, so reads always hit the cache. Option 2 is cleanest for read-heavy workloads that have occasional writes.

How do you design a RAG system for 10 M documents?

Offline: chunk documents (256–512 tokens, 10% overlap) → embed with a sentence-transformer → store vectors + metadata in a dedicated vector DB (Qdrant/Weaviate with HNSW index). At query time: embed user query → ANN search (top-100) → BM25 keyword search → Reciprocal Rank Fusion → cross-encoder re-rank top-10 → inject into LLM context. Scale: dedicated embedding microservice (GPU or CPU depending on model size); vector DB sharded by document collection; KV cache for repeated system-prompt prefixes in vLLM.

What would you monitor in production for a system you just designed?

Four golden signals: Latency (p50/p99/p999), Traffic (QPS by endpoint), Errors (5xx rate, % of requests), Saturation (CPU/memory/disk, queue depth, DB connection pool). Plus: cache hit rate (if <80%, investigate), replication lag (if >1 s, investigate), queue consumer lag (Kafka: lag by consumer group), and business metrics (revenue, orders/min). Alert on anomaly detection (sudden drop in traffic = bad as much as sudden spike).

> **How does Kafka guarantee message ordering?**
>
> Kafka guarantees ordering within a partition. Messages with the same key always go to the same partition (key-based partitioning), so all events for a given entity (e.g. user_id=123) are ordered. Across partitions there is no total order — only per-partition ordering. For global order you would need a single partition (sacrificing parallelism) or a coordination layer. Consumer groups scale up to the partition count; adding more consumers than partitions yields idle consumers.

How to Communicate Your Design

Technical correctness is necessary but not sufficient. How you communicate is evaluated as heavily as what you design.

Narrate, Don't Draw in Silence

Speak every decision as you make it: "I'm adding a Redis cache here because reads are 100× more frequent than writes and DB latency would otherwise dominate." Silence reads as uncertainty. Narration shows reasoning, not just recall.

Check In Before Going Deep

After the high-level sketch, pause: "Does this structure look right to you? Is there a specific component you'd like me to go deeper on?" This prevents 20 minutes on the wrong deep-dive. It also shows collaborative instinct — a senior engineer would seek alignment before over-engineering one corner.

Lead with Trade-offs

Never present a decision without its cost: "I'm using eventual consistency here which means a user might see a stale count for a few seconds — that's acceptable for a social feed but I'd reconsider for financial transactions." Interviewers at senior levels are specifically listening for this vocabulary.

Name What You Are Not Solving

Explicitly park scope: "I'm not designing the recommendation engine today — that's a separate ML problem. I'll note it as a future extension." This demonstrates scope awareness and prevents rambling.

Handle Pushback Gracefully

When the interviewer challenges a decision, say: "That's a good point. If [concern] is true, the better approach would be [alternative] because [reason]. I chose the original approach assuming [assumption] — want me to redesign with your assumption?" This models how senior engineers actually change their minds.

✓ **Golden rule**

Drive the conversation. The best candidates never wait to be asked the next question — they say "Now that we have the high-level design, let me walk through the most interesting problem: how do we handle hot-key contention in the cache." Ownership of the narrative is itself a signal of senior-level behaviour.

Common Mistakes to Avoid

JUMPING TO SOLUTIONS

Saying "I'll use Kafka" before establishing the scale or requirement signals pattern-matching rather than reasoning. State the requirement first, then justify the choice.

IGNORING FAILURE MODES

Every component you add is a potential failure point. For each box you draw, briefly mention: what happens when it goes down? What is the fallback? What is the SLA impact?

PREMATURE OPTIMISATION

Designing for 1 billion users when the requirement is 10 million wastes interview time and signals poor prioritisation. Scale to the stated requirement; note what you'd change at 100×.

NO ESTIMATION

Skipping back-of-envelope math leaves your design ungrounded. Estimation reveals whether you need 1 server or 1 000. Always quantify before architecting.

30-minute interview pacing guide